



Technical Assessment – July 2026

Porua AI – Backend

Build a small backend for an AI-powered study app, Porua. The scope is deliberately open-ended and a bit narrow so you can go deep on the parts that matter: the API shape, the data model, and the messy edges of working with an LLM.

The product

A student creates a room for a topic such as **AP Biology: Cell Respiration** or **Grade 7 Algebra: Solving Equations** and has short conversations with an AI tutor. They can ask for explanations, upload study materials as context, and view history. When they return to a specific room, the conversation continues from relevant earlier context.

Over time, the student builds a queryable history of what they studied and what happened in those sessions.

What to build

Design and implement the API using Python and FastAPI. Endpoint design, database, libraries, and project structure are your choice.

You will be provided a Gemini API key and the fixed model is *gemini-2.5-flash*.

We should be able to:

- Create and list study rooms
- Open a room and see what happened inside it
- Start a new AI turn/chat
- Inspect a turn in enough detail to understand its prompt, model attempts, tool activity, failures, (bonus) token usage data, and final saved result
 - Data must persist between runs.
- Query a student's study history over a period of time
- Upload Word, PDF, and PowerPoint files as source materials in the room as context. The AI tutor can query those as needed

Note: A login flow is not required.

Data design

Use a relational database. We care about real relationships, appropriate constraints and indexes, and how the design behaves as history grows.

Skills and Tools

Power the agent with useful **tools** (at least 2) and study **skills** (at least 2) of your own – for example, past chat query tool, a step-by-step math explainer skill, etc.

The new **skills** should provide real student value, extend the LLM's capabilities, and not be renamed versions of the same prompt.

Prompt engineering and AI reliability

Use the LLM APIs wisely. Prompt engineering is a first-class part of the assignment. We will evaluate whether the output is grade-appropriate, accurate, focused, and genuinely useful. Keep prompts readable and versioned.

Assume the Gemini model will sometimes ignore instructions, return malformed data, or produce structurally valid but poor content. Build enough reliability around it that you would be comfortable exposing the result through an API.

For example, a quiz might contain duplicate choices, the wrong count, an invalid answer, or predictable answer option patterns. Decide how to overcome LLM shortcomings/non-determinism.

Tool use must be controlled; the model should not call arbitrary code or call tools indefinitely. How you solve these problems, and which others you identify, is up to you.

Document processing

The student must be able to use these common school files as source material:

- Word (.docx)
- PDF (.pdf)
- PowerPoint (.pptx)

Build a real ingestion path that extracts accurate content. Handle bad and unsupported inputs deliberately.

Deliverables

- A private GitHub repository
 - Add [ikramhasan](#) and [azmainadel](#) as collaborators
 - Archive the repository to make it read-only
- A README.md with setup instructions, important decisions, prompt notes, known limitations, and whatever you think is worth telling us
- Usable Swagger documentation; a small web interface is optional
- Codebase should include:
 - Database migration files
 - Readable prompts in the repository
 - An example environment file containing placeholder values
 - A progressive commit history
 - (Optional) tests and fixtures.

If you make an assumption, document it.

If something essential is unclear, ask us without hesitation. Reply to the email.

What we value

We assume you will use AI coding tools, we would too. The bar is not what you can ship in the time we've given, it is the shape of what you ship.

What you noticed on your own, what you chose to prioritize, what you deliberately left alone, and whether the whole thing hangs together as one considered piece of work.

We walk through the solution together on the review call. Bring what you would have done with more time and any decisions you want to defend.

Deadline

22 July, 2026 - 6AM

After submission, we will book a review call.

You are not expected to complete every part perfectly. Prioritize thoughtfully and get as far as you can. The scope is deliberately open-ended so we can see your judgment, trade-offs, and approach.

